SECTION 2 | LECTURE 6

수업 6. 이 앱에는 어떤 문제가 숨어 있는가

코드를 고치기 전에 문제를 먼저 드러냅니다—Part 1에서는 공통 baseline 애플리케이션을 통해 비용, 품질, 리스크, 흐름을 분해하고, 미리 작성된 기준 코드의 흐름을 전체적으로 파악하는 데 집중합니다.

데모와 운영의 차이: Demo-ready ≠ Ops-ready

DEMO PERSPECTIVE

데모 관점: 잘 동작하는 baseline

입력을 넣으면 오류 없이 그럴듯한 결과물이 즉시 출력됨

- **응답 수신 정상:** Incident 입력 시 HTTP 200 성공 응답 반환
- **내용의 설득력:** 원인 분석과 대응 액션 텍스트가 제법 그럴듯함
- **예외 미발생:** 실행 중 런타임 셧다운이나 에러 없이 수월하게 종료

"시연에서는 아무런 문제가 없어 보임"

OPERATIONAL PERSPECTIVE

운영 관점: 숨겨진 4가지 물음표

실제 프로덕션 환경에 투입했을 때 마주할 가혹한 현실 질문들

- **[비용]** 이 호출 한 건에 토큰과 요금이 얼마나 드는가?
- **[품질]** 생성된 분석과 가설을 정말로 신뢰할 수 있는가?
- **[리스크]** 추천된 인프라 명령어를 그대로 실행해도 안전한가?
- **[흐름]** 이 요청이 내부에서 정확히 어떤 경로로 흘렀는가?

Demo-ready ≠ Ops-ready (운영적 통제의 필요성)

오늘 코드를 보는 방식: Mark it, don't fix it yet

RECOMMENDED MINDSET

DO : 문제지점 마킹 및 한 줄 진단

통제 4축 관점으로 소스 코드를 읽고 의심 지점에 레이블 남기기

- **통제 4축 관점으로 소스 코드 읽기:** 비용, 품질, 리스크, 흐름 관점으로 라인 독해
- **위치 마킹:** 의심 지점에 미리 작성된 주석 위치 확인
- **이유 명시:** "왜 이 코드가 운영상 위험한가"를 한 줄로 정리

"Mark it, don't fix it yet"

AVOID TODAY

DON'T : 즉각적인 코드 수정 및 구현

문제의 정의와 범위가 명확해지기도 전에 성급하게 고치기

- **성급한 리팩터링:** 로직 수정이나 클래스 구조 변경 시도 금지
- **검증 로직 추가:** 지금 당장 DTO 검증이나 Advisor 작성 금지
- **즉시 해결 시도:** 문제가 정확히 드러나기도 전에 조급히 수정하기

"해결책은 수업 8~16에서 각 축별로 순서대로 전개됩니다"

문제 매핑: 이 문제는 코드의 어디에 있는가

CONTROL AXIS	MAPPED JAVA CLASSES	ROOT CAUSE IN CODE
COST	IncidentPromptFactory LocalRunbookKnowledgeAdapter LocalIncidentHistoryAdapter	매 호출마다 runbook, history, policy 전체를 요약 없이 프롬프트에 통째로 바인딩
QUALITY	IncidentAiAnalysisOutput IncidentAiActionOutput PromptFormatUtils IncidentChatGateway IncidentAnalysisService IncidentActionRecommendationService	"JSON만 반환하라"는 자연어 지시뿐, 강제 스키마 검증기나 재시도/Fallback 루프 부재
RISK	IncidentActionRecommendationService LocalAlertPolicyAdapter	파괴적 인프라 액션을 검증/승인 게이트 없이 단순 텍스트로 응답 객체에 직접 전달
FLOW	IncidentResponderService TraceLoggingFilter AiObservationSupport	분기 없는 일직선 파이프라인이며, 관찰 도구(`AiObservationSupport`)가 실코드에서 미호출됨

문제점들은 막연히 존재하는 것이 아니라 특정 클래스와 메서드 라인 위에서 정확히 포착됩니다.

오늘 정리 & 다음 수업 안내

TODAY SUMMARY

통제 관점 코드 진단

- **Demo vs Ops 격차 확인:** 정상 동작하는 데모 이면에 숨은 운영 위험 식별
- **4축 문제 매핑 완료:** 비용(프롬프트), 품질(미검증), 리스크(직출력), 흐름(미관찰) 매핑
- **문제지점 가시화:** 사전 마킹된 주석을 짚어가며 코드 수준의 문제 위치 고정

#code_review #control_4_axes #problem_identification

UP NEXT LECTURE

다음 수업 미리보기 (수업 7)

오늘 확인한 문제 지점들을 이제 실제 관찰 가능한 흐름으로 연결합니다.

LECTURE 7 TOPIC

Spring AI Observability 연결하기 – 샘플 앱에 계측 추가

TraceLoggingFilter → Micrometer Tracing → Spring AI Observations → Langfuse로 이어지는 흐름 통제 출발점 수립

Move to Next Lecture